Pilgrim Engine - Game State Representation

This document explains how the Pilgrim rules engine represents game state, how scenario files relate to that state, and how the CLI commands can be used to inspect and advance a position.

Core Mental Model

The engine represents the game as a deterministic snapshot of the whole board at one exact moment.

```
GameState = everything needed to answer:
1. Whose turn is it?
2. What resources, workforce, and buildings does each player have?
3. What is on the shared board?
4. What phase, round, and season are we in?
5. What actions are legal from here?
6. What happens if one action is applied?
```

The engine uses the current GameState to generate legal actions, then applies one selected action to produce a new GameState and a list of events.

```
legal_actions(GameState) -> list of legal actions
apply_action(GameState, action) -> new GameState + events
```

A scenario file is therefore a serialized board position. It describes an initial GameState that can be validated, inspected with legal-actions, or advanced with apply.

Core State Categories

Timing and Turn State

Timing fields describe where the game is in the turn, round, and season structure.

```
absolute_turn
round_number
season
turn_in_round
phase
game_over
```

Turn order is controlled by:

```
active_player
start_player
player_count / real players
```

The distinction is important:

```
active_player = the player who acts now
start_player = the first-player marker / who starts the round
turn_in_round = how far through the current round we are
```

In normal play, each real player acts once per round. After the last player in the round acts, the round-end pipeline runs. After round end, start_player and active_player are updated according to the start-player selection policy.

Player State

Each player has their own local board state. This includes resources, tracks, workforce, and player board slots.

```
resources:
  stone
  silver
```

```
  wheat

tracks:
  piety_position
  alms_position

workforce:
  mancala positions
  village
  abbey
  special activities
  committed acolytes

player board slots:
  active buildings
  donated buildings
  cardinal favor tiles
```

Examples:

```
Pulpit changes workforce by moving one serf from Village to Abbey.
Grain Store changes wheat and silver.
Indulgences changes piety and silver.
Alms season-end scoring moves one Abbey acolyte to the Alms Table, if available.
Round-end excess capping may reduce stone and wheat to the maximum allowed store value.
```

Shared Board State

The shared board state contains information that applies to all players.

```
duty tiles by board position
merchant position
ship position
building market
building availability
dummy acolytes
pilgrimage rounds / setup metadata
```

The duty tile layout tells the engine what each board position currently represents. For example:

```
north = produce
north_east = clerical
east = build_roads
south_east = construct
south = give_alms
south_west = ordination
west = allocation
north_west = taxation
```

The Merchant position determines the current hire resource. Building availability determines which market buildings are live. Pilgrimage round metadata determines when season-end Alms scoring triggers.

Buildings and Action Modifiers

Buildings are represented in the state through ownership, market presence, and availability.

```
A building may be:
- owned and active
- donated
- in the market and live
- in the market but not live yet
- owned by another player and hireable
- unavailable
```

The state stores which buildings exist and where they are. The action stores whether a building effect is being used on that particular turn.

```
Examples of building-use metadata on actions:

building_conversion_id = grain_store / indulgences / stone_yard / brewery
merchant_advance_building_id = guild
workforce_move_building_id = pulpit
sow_route_building_id = kogge / cloisters
start_turn_building_id = dormitory / inquisition
end_turn_building_id = library
```

This separation is useful:

```
GameState = what is available
Action = what is chosen this turn
```

For example, a player may own active Pulpit, but a given action may or may not use Pulpit. Legal action generation creates both the normal action and the Pulpit-modified variant when the Pulpit effect is legal.

Legal Action Generation

The engine generates legal actions from the current state. It checks things such as:

```
where the active player's acolytes are
which sow routes are legal
which Duty tiles can be selected
which Duty actions are available
which buildings are usable
whether hire costs can be paid
whether resource or workforce costs can be paid
whether optional modifiers can be applied
```

For example, in a Pulpit scenario where player_one owns active Pulpit and has one acolyte on north, legal actions might include:

```
1. normal Clerical action
2. normal Clerical action
3. normal Clerical tithe
4. Pulpit + Clerical action
5. Pulpit + Clerical action
6. Pulpit + Clerical tithe
```

This means Pulpit is not a standalone action. It is an optional modifier attached to otherwise legal full-turn actions.

Applying an Action

Applying an action simulates choosing one item from the legal action list. The engine produces:

```
events
new GameState
invariant check
```

Events explain what happened. The new state reflects the result. For example, applying a Pulpit action may produce:

```
BUILDING_BONUS
WORKFORCE_MOVE
SOWING
DUTY_RESOLUTION
TURN_ADVANCE
INVARIANT_CHECK
```

The resulting state then shows the updated workforce and the updated mancala state from sowing.

```
Village: 2 -> 1
Abbey: 0 -> 1
```

Scenario Files as Board Positions

Scenario files are not only tests. They are serialized board states. A scenario can be used to:

```
validate that the state loads correctly
list all legal actions from that state
apply one legal action and inspect the result
serve as a focused regression fixture
serve as an input to search/optimization
```

Typical CLI workflow:

```
python3 -m pilgrim.cli validate scenarios/example.json
python3 -m pilgrim.cli legal-actions scenarios/example.json
python3 -m pilgrim.cli apply scenarios/example.json --action-index 1 --verbose
```

legal-actions prints the menu of legal choices from the scenario state. apply selects one numbered action from that menu, applies it, and prints the resulting events and state.

The --action-index value is the numbered action from the legal-actions output. It is not a Duty tile number.

Why This Representation Works for Search

This representation is useful for optimization because the engine can repeatedly perform:

```
state
-> legal_actions(state)
-> choose one action
-> apply_action(state, action)
-> next_state
-> legal_actions(next_state)
-> ...
```

This is the foundation for exhaustive search, heuristic search, branching audits, and future optimization methods.

The key design principle is:

```
GameState should contain everything needed to compute legal actions and apply transitions deterministically.
```

Actions are explicit, deterministic choices. Events explain what changed. The next GameState is the new board position.